

Amlogic NPU Quick Start Guide

Version: v0.1

Release Date: 2026-06-25

版权

© 2026 晶晨半导体（上海）股份有限公司（代表其自身及其关联公司，以下简称“Amlogic”）版权所有。未经 Amlogic 书面许可，不得以任何形式或通过任何手段对本文件的任何部分进行复制、传播、转录或翻译成任何语言。

商标

 Amlogic，及 Amlogic 的其他图标均为 Amlogic 公司的商标。所有其他商标和注册商标均为其各自所有人的财产。

免责声明

由于产品版本升级或其他原因，本文档将可能在未经任何通知的情况下，不定期进行更新或修改。

本产品不适用于医疗、救生或生命维持等用途，也不适用于可合理预见因 Amlogic 产品故障或失灵可能导致人身伤害、死亡、严重财产或环境损害的应用场景。

文中提供的电路图及其他与 Amlogic 产品相关的信息仅为典型应用说明，完整且足以用于生产设计的全部信息不一定会提供。Amlogic 已尽量确保本文件内容准确和及时，但 Amlogic 不对本文档所含信息的准确性或完整性作任何保证。

Amlogic 产品可能存在未被识别的漏洞，或可能支持具有已知局限性的既定安全标准或规范。Amlogic 对任何漏洞不承担任何责任。

使用相关材料可能需获得第三方或 Amlogic 的知识产权许可。本文档对归属于 Amlogic 或任何其他方的知识产权不授予任何明示或暗示的声明或保证。文中包含的任何第三方网站链接仅为方便用户而提供，Amlogic 不对该等网站及其任何行为（包括隐私惯例、可用性和内容）作出认可，也不承担任何责任。

在任何情况下，Amlogic 均不对与本文档相关、由本文档使用或无法使用所产生的任何索赔承担责任。

联系方式

- 公司网址: www.amlogic.cn
- 售前咨询: contact@amlogic.com
- 技术支持: support@amlogic.com

修订记录

版本	修改说明
v0.1 (2026-06-25)	初始版本

目录

1 文档说明

- 1.1 文档目的
- 1.2 适用平台
- 1.3 关键字说明

2 开发环境确认

- 2.1 版本确认
- 2.2 系统环境与架构确认
- 2.3 快速验证
 - 2.3.1 通用Case验证
 - 2.3.2 LLM 快速验证

3 Toolkit 安装

- 3.1 部署方式与工具包简介
- 3.2 下载AMLNN相关仓
 - 3.2.1 amlnn-toolkit介绍
 - 3.2.2 amlnn-model-playground 介绍
- 3.3安装Toolkit环境
 - 3.3.1 Ubuntu PC 端 amlnn-toolkit 安装
 - 3.3.2 板端 Armbian 系统安装 amlnn-edge-toolkit

4 板端转换推理

- 4.1 前期准备
- 4.2 模型转换与推理
- 4.3 USB摄像头实时采集Demo获取指南

5 PC端模型转换

- 5.1 前期准备
 - 5.1.1 下载模型
 - 5.1.2 平台类型确认
- 5.2 模型转换

6 ADB连板推理

- 6.1 安装ADB工具
- 6.2 启动nnserver
- 6.3 运行 AMLNN Python Demo

7 板端 C/C++ 推理

- 7.1 安装编译工具链
 - 7.1.1安装Cmake
 - 7.1.2安装交叉编译工具链
 - 7.1.2.1 Android系统：安装与配置NDK
 - 7.1.2.2 Buildroot系统：安装 GCC 交叉编译工具链
 - 7.1.2.3 Armbian/Yocto系统：安装交叉编译工具链
- 7.2 运行 AMLNN C/C++ demo
 - 7.2.1 Android系统C/C++ demo 编译运行
 - 7.2.2 Buildroot系统C/C++ demo 编译运行
 - 7.2.3 Armbian/Yocto系统C/C++ Demo编译运行
 - 7.2.4 预期结果

8 常见问题

- 8.1 adb devices 无法识别设备
- 8.2 nnserver 服务相关问题

1 文档说明

1.1 文档目的

本文档旨在帮助开发者快速上手 Amlogic NPU(ADLA(amlogic Deep Learning Accelerator))平台的深度学习模型部署。无论您是初次接触嵌入式 AI 开发，还是希望将现有模型迁移到 Amlogic 平台，本指南都将为您提供从环境搭建、模型转换、编译部署到运行验证的完整流程。

本文档所用示例已集成到**AMLNN Model Playground**中。

- **AMLNN Toolkit**: Amlogic NPU 开发的核心枢纽，涵盖了在 Amlogic 平台上部署 AI 模型所需的完整软件栈——从高层 Python 运行时到底层 C/C++ SDK 及参考示例。
- **AMLNN Model Playground**: 是基于 **AMLNN Toolkit** 构建的模型库，提供了丰富的主流算法示例，包括完整的模型转换脚本以及使用 Python API、C/C++ API 进行推理的完整代码。

1.2 适用平台

本指南适用于以下 Amlogic NPU 平台：

- C308L / C302X
- S928X
- A311D2
- T968D4
- S905X5 / S905D5
- C302X2
- C305X2/A123X
- A311Y3

各平台支持的操作系统：

	Buildroot	Android	Armbian(Debian)	Yocto
C308L/C302X	✓	×	×	×
S928X	×	✓	×	✓
A311D2	×	✓	×	✓
T968D4	×	✓	×	✓
S905X5/S905D5	×	✓	×	✓
C302X2	✓	×	×	×
A311Y3	×	✓	✓	✓
C305X2/A123X	✓	×	×	×

1.3 关键字说明

- **ADLA**: ADLA(amlogic Deep Learning Accelerator)，运行在amlogic 平台上的模型文件，后缀为.adla。
- **Armbian (Debian)**：后续文档中统一简称为 Armbian。

2 开发环境确认

2.1 版本确认

确认 ADLA NPU 驱动版本及 AML Runtime 库版本是否满足需求。通过 adb 连接开发板后，按以下操作确认版本情况。

Step1: NPU驱动版本确认

- 执行命令：adb shell cat /sys/class/adla/adla0/device/kmd_version
- 参考结果如下：

```
a9_a311y3:/ # cat /sys/class/adla/adla0/device/kmd_version
ADLA Version: 2.0.2.0.0
a9_a311y3:/ # |
```

- 预期结果：Version >=2.0.2

Step2: NNSDK版本确认

- 执行命令

```
adb shell

# for Android
strings /vendor/lib64/libnnsdk.so | grep NNSDK

# for Linux(buildroot/armbian/yocto)
strings /usr/lib/libnnsdk.so | grep NNSDK
```

- 参考结果如下：

```
a9_a311y3:/ # strings /vendor/lib64/libnnsdk.so | grep NNSDK
NNSDK Version: v3.0.0, 2026.05
NNSDK2 Version: v1.0.0, 2026.05
vendor.NNSDK2_LOG_LEVEL
NNSDK2
a9_a311y3:/ # |
```

- 预期结果：
 - NNSDK Version >=3.0.0
 - NNSDK2 Version >=1.0.0

2.2 系统环境与架构确认

确认板端系统环境与架构，旨在为后续章节选取对应系统的快速开始测试文件、nnserver 及交叉编译工具链提供指导

Step1 先确认是Android平台或者Linux平台

执行命令：adb shell getprop ro.product.cpu.abi

判定逻辑：

- **输出** `arm64-v8a` : 则确认为 64 位 Android 系统
- **输出** `armeabi-v7a` : 则确认为 32 位 Android 系统
- **报错** `command not found` : 判定为 Linux 系统, 请继续执行 Step 2

Step2 Linux 系统 (Buildroot / Yocto / Armbian) 环境进一步确认

Step2.1 确认是否为 Armbian / Yocto 系统

执行命令: `cat /version.txt`

参考输出示例(以 Armbian 为例)

```
image: a9-by401-bookworm-20260707030949
BRANCH=
VERSION=Linux/Debian/Armbian-PB#786
SPIN=Linux/Debian/Armbian-PB#786
BUILD_TIME="2026-07-07 03:27:41"
Generated on Tue Jul 7 03:27:41 UTC 2026

Compiler:
aarch64-none-linux-gnu-gcc (Arm GNU Toolchain 11.3.Rel1) 11.3.1 20220712
Copyright (C) 2021 Free Software Foundation, Inc.
This is free software; see the source for copying conditions. There is NO
warranty; not even for MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.
```

判定逻辑:

- **有该文件存在:** 则判断为 armbian/Yocto 系统, 再通过 Compiler 版本信息判断是 32 位/64 位系统。参考示例对应 **Armbian 64-bit (AArch64)** 系统
- **文件不存在:** 命令执行失败, 判断为 Buildroot 系统, 请继续执行 Step 2.2

注意: 上述确认方式仅适用于 Amlogic 官方 Release 版本单板。第三方开发板请自行确认是否支持相同的版本文件与判定逻辑。

Step2.2 确认 Buildroot 系统架构

执行命令: `adb shell uname -m`

判定逻辑:

- `aarch64` : 则确认为 64 位 buildroot 系统
- `armv7l` : 则确认为 32 位 buildroot 系统

2.3 快速验证

根据不同单板及系统, 选择对应平台的可执行文件与 ADLA 文件, 快速验证 NPU 运行环境是否正常。

2.3.1 通用 Case 验证

当前章节的快速验证方法仅适用于本文档所列的全部 Amlogic NPU 单板, 且要求这些单板的系统环境已通过第 2.1 章节的版本确认。

Step1: 获取 native case

下载 link: https://pub-8378326bd0fe4b1d9312a3847f6316a2.r2.dev/Quick_Start/DDK5.10.4.10/ddk_5.10.4.10_native_case.tar.gz

下载解压后，目录文件如下

```
├─ 224x224x3_fish_uint8.bin
├─ mobilenet_v1_1.0_224_quant_a311d2.adla
├─ mobilenet_v1_1.0_224_quant_a311y3.adla
├─ mobilenet_v1_1.0_224_quant_c302x2.adla
├─ mobilenet_v1_1.0_224_quant_c305x2.adla
├─ mobilenet_v1_1.0_224_quant_c308l.adla
├─ mobilenet_v1_1.0_224_quant_s905x5.adla
├─ mobilenet_v1_1.0_224_quant_s928x.adla
├─ mobilenet_v1_1.0_224_quant_t968d4.adla
├─ nnsample_android_32
├─ nnsample_android_64
├─ nnsample_buildroot_32
├─ nnsample_buildroot_64
├─ nnsample_yocto_32
└─ nnsample_yocto_64
```

Step2: 运行native case

Android系统:

Step1:

将对应可执行bin文件和对应平台adla，push到/data/local/tmp目录

Step2:

#64位系统

```
adb shell
```

```
cd /data/local/tmp
```

```
chmod +x nnsample_android_64
```

```
./nnsample_android_64 mobilenet_v1_1.0_224_quant_a311y3.adla
```

```
224x224x3_fish_uint8.bin --base
```

#32位系统

```
adb shell
```

```
cd /data/local/tmp
```

```
chmod +x nnsample_android_32
```

```
./nnsample_android_32 mobilenet_v1_1.0_224_quant_a311y3.adla
```

```
224x224x3_fish_uint8.bin --base
```

Buildroot系统:

Step1:

将对应可执行bin文件和对应平台adla，push到/data/local/tmp目录

Step2:

#64位系统

```
adb shell
```

```
cd /data/local/tmp
```

```
chmod +x nnsample_buildroot_64
```

```
./nnsample_buildroot_64 mobilenet_v1_1.0_224_quant_a311y3.adla
```

```
224x224x3_fish_uint8.bin --base
```

#32位系统

```
adb shell
```

```
cd /data/local/tmp
```

```
chmod +x nnsample_android_32
./nnsample_buildroot_32 mobilenet_v1_1.0_224_quant_a311y3.adla
224x224x3_fish_uint8.bin --base
```

Yocto/Armbian系统:

Step1:

将对应可执行bin文件和对应平台adla，push到/data/local/tmp目录

Step2:

#64位系统

```
adb shell
```

```
cd /data/local/tmp
```

```
chmod +x nnsample_yocto_64
```

```
./nnsample_yocto_64 mobilenet_v1_1.0_224_quant_a311y3.adla
```

```
224x224x3_fish_uint8.bin --base
```

#32位系统

```
adb shell
```

```
cd /data/local/tmp
```

```
chmod +x nnsample_yocto_32
```

```
./nnsample_yocto_32 mobilenet_v1_1.0_224_quant_a311y3.adla
```

```
224x224x3_fish_uint8.bin --base
```

参考结果:

```
2: 0.996094
0: 0.000000
1: 0.000000
3: 0.000000
4: 0.000000
```

确认运行在NPU上:

- Step1: 推理前后分别执行命令: cat /proc/interrupts | grep adla
- Step2: 两次命令执行结果中，第二列对应的中断数数量有增加。

```
/data/# cat /proc/interrupts | grep adla
91:          514          0          0          0          0 adla
```

2.3.2 LLM 快速验证

当前章节所述的快速验证方式，仅适用于 A311Y3 平台所支持的系统环境。

Step1:获取LLM Native Case:

LLM_Native_case下载link: https://pub-8378326bd0fe4b1d9312a3847f6316a2.r2.dev/Quick_Start/DDK5.10.4.10/ddk_5.10.4.10_llm_native_case.tar.gz

下载解压后，目录文件如下：

```
├─ llm_demo_android_32
├─ llm_demo_android_64
├─ llm_demo_yocto_32
├─ llm_demo_yocto_64
└─ Qwen1.5-0.5B-Chat-Q4AM_PB32.adla
```

Step2: 运行 LLM Native Case:

Android系统:

Step1:

将对应可执行bin文件和对应平台adla，push到/data/local/tmp目录

Step2:

#64位系统

```
adb shell
```

```
cd /data/local/tmp
```

```
chmod +x llm_demo_android_64
```

```
./llm_demo_android_64 Qwen1.5-0.5B-Chat-Q4AM_PB32.adla --model_type qwen
```

#32位系统

```
adb shell
```

```
cd /data/local/tmp
```

```
chmod +x llm_demo_android_32
```

```
./llm_demo_android_32 Qwen1.5-0.5B-Chat-Q4AM_PB32.adla --model_type qwen
```

按照提示，可以做问答

Yocto/Armbian系统:

Step1:

将对应可执行bin文件和对应平台adla，push到/data/local/tmp目录

Step2:

#64位系统

```
adb shell
```

```
cd /data/local/tmp
```

```
chmod +x llm_demo_yocto_64
```

```
./llm_demo_yocto_64 Qwen1.5-0.5B-Chat-Q4AM_PB32.adla --model_type qwen
```

#32位系统

```
adb shell
```

```
cd /data/local/tmp
```

```
chmod +x llm_demo_yocto_32
```

```
./llm_demo_yocto_32 Qwen1.5-0.5B-Chat-Q4AM_PB32.adla --model_type qwen
```

按照提示，可以做问答

3 Toolkit 安装

3.1 部署方式与工具包简介

本章将系统介绍不同板端系统所支持的部署方式及配套工具包。目前主要涵盖以下几种推理方式（当前文档将详细介绍**方式一**、**方式二**、**方式四**的具体操作流程）

方式一：PC端模型转换 + ADB连板推理（快速验证）

- **流程**：Ubuntu PC 通过 `amlnn_toolkit` 转出 ADLA --> 通过 ADB 协议远程调用板端 NPU 完成推理
- **使用**：本文档覆盖的全系列单板，需Ubuntu主机与ADB连接
- **场景**：算法调试、模型效果快速预览

方式二：PC端模型转换 + 板端 C/C++ 推理（生产级高性能部署）

- **流程**：Ubuntu PC 通过 `amlnn_toolkit` 转出 ADLA --> 业务代码 C/C++交叉编译 -->板端调用 AML Runtime推理
- **适用**：本文档覆盖的全系列单板，需Ubuntu主机与ADB连接（传输文件）
- **场景**：高实时性、无Python依赖的生产环境

方式三：PC端模型转换 + 板端Lite Python推理（Armbian/Yocto轻量开发）

- **流程**：Ubuntu PC 通过 `amlnn_toolkit` 转出 ADLA --> 板端部署 `amlnn_edge_toolkit_lite` --> Python API推理
- **适用**：本文档覆盖的搭载 Armbian/Yocto 系统的 Amlogic NPU 单板，需Ubuntu主机与对应开发板
- **场景**：Python快速开发，免交叉编译

方式四：板端转换推理（无PC依赖的边缘离线方案）

- **流程**：板端安装完整 `amlnn_edge_toolkit` -->本地转换+推理
- **适用**：本文档覆盖的搭载 Armbian/Yocto 系统的 Amlogic NPU 单板（Yocto需额外补全Python及开发库依赖）
- **场景**：离线边缘部署、现场模型迭代

上面介绍的四种Amlogic NPU推理部署方案中，所用到的三款核心Python Whl工具包，对应介绍如下：

`amlnn_toolkit`

- **简介**：PC 端模型转换工具，支持通过 adb 连接板端进行推理
- **运行环境**：Ubuntu PC

`amlnn_edge_toolkit`

- **简介**：面向 armbian/ Yocto 板端系统的模型转换与推理工具。受限于板端环境，仅支持部分模型的转换（大模型转换会出现内存不足异常），并提供 Python 接口直接推理。
- **运行环境**：搭载 Armbian/Yocto 系统的 Amlogic NPU

`amlnn_edge_toolkit_lite`

- **简介**：面向 Debian / Yocto 板端系统的 ADLA 推理工具
- **运行环境**：搭载 Armbian/Yocto 系统的 Amlogic NPU

3.2 下载AMLNN相关仓

3.1 节介绍的四种推理方式中，**方式一、方式二、方式三**均需在 PC 端完成模型转换，因此需准备一台 Ubuntu 20.04 / 22.04 主机，并直接在该环境下下载 AMLNN 相关仓库。

方式四则无需 PC 转换环境，其转换与推理均在板端完成，仅需通过 GitHub 链接下载所需资源即可，无需在板端系统拉取 AMLNN 相关仓库。

AMLNN 相关仓库下载参考命令如下：

```
# 创建工程目录
mkdir ~/workspace
cd ~/workspace

# 克隆amlnn-toolkit示例工程仓库
git clone https://github.com/Amlogic-NN/amlnn-toolkit.git --depth 1

# 克隆AMLNN示例工程仓库
git clone https://github.com/Amlogic-NN/amlnn-model-playground.git --depth 1
```

建议在 PC 端统一将上述仓库存放于工作目录（如 `~/workspace/`）下，便于后续模型转换与示例运行。

3.2.1 amlnn-toolkit介绍

整体目录结构如下：

```
└─ amlnn-toolkit
  │   └─ amlnn_runtime          # runtime so
  │   └─ amlnn_toolkit
  │       └─ nnserver          # nnserver bin file
  │       └─ whl
  │           └─ aarch64      #amlnn_edge_toolkit whl package
  │           └─ linux_x86    #amlnn_toolkit whl package
  │   └─ amlnn_toolkit_lite
  │       └─ whl              # amlnn_edge_toolkit_lite whl package
  │   └─ doc                  # AMLNN documentation
  │   └─ ...
```

3.2.2 amlnn-model-playground 介绍

整体目录结构如下：

```
└─ assets
└─ cmake
└─ common
└─ dependency
└─ docs
└─ examples
  │   └─ whisper
  │       └─ cpp
  │           └─ build-android.sh    #Android 系统开发板编译脚本
  │           └─ build-linux.sh      #Linux 系统开发板编译脚本
  │           └─ src                  # C++ 示例代码
  │       └─ input
  │       └─ model
```



```
# 确保已激活 amlnn 环境
conda activate amlnn

# 进入 amlnn-toolkit 目录
cd ~/workspace/amlnn-toolkit

# 安装 AMLNN-Toolkit 依赖及包
for req in $(cat amlnn_toolkit/whl/linux_x86/requirements_x86_64.txt); do pip
install $req; done
pip install amlnn_toolkit/whl/linux_x86/amlnn_toolkit-1.0.0-cp310-cp310-
linux_x86_64.whl
```

Step4:验证安装

```
python -c "from amlnn.api import AMLNN; print('AMLNN-Toolkit import OK')"
```

#成功打印 AMLNN-Toolkit import OK

3.3.2 板端 Armbian 系统安装 amlnn-edge-toolkit

针对需在板端直接完成模型转换与推理的场景，请按照以下步骤在 Armbian 系统中安装 `amlnn-edge-toolkit`。为节省板端存储空间，请直接从 [AMLNN Toolkit](#) 仓库的 `amlnn_toolkit/whl/aarch64/` 目录下载预编译文件，具体包括：

- `requirements_aarch64.txt`
- `amlnn_edge_toolkit-1.0.0-cp310-cp310-linux_aarch64.whl`

下载完成后，执行以下命令进行安装：

Step1:安装Miniforge

```
wget https://github.com/conda-
forge/miniforge/releases/latest/download/Miniforge3-Linux-aarch64.sh

bash Miniforge3-Linux-aarch64.sh

# 按照提示操作，完成后重新加载配置
source ~/.bashrc
```

Step2:创建并激活Python虚拟环境

```
conda create -n amlnn python=3.10 -y
conda activate amlnn

python --version
pip install --upgrade pip
```

Step3:安装amlnn_edge_toolkit

```
# 确保已激活 amlnn 环境  
conda activate amlnn
```

```
# 安装 AMLNN-Toolkit 依赖及包  
for req in $(cat requirements_aarch64.txt); do pip install $req; done  
pip install amlnn_edge_toolkit-1.0.0-cp310-cp310-linux_aarch64.whl
```

Step4:验证安装

```
python -c "from amlnn.api import AMLNN; print('AMLNN-Toolkit import OK')"  
  
#成功打印 AMLNN-Toolkit import OK
```

4 板端转换推理

当前章节适用于本文档覆盖的搭载 Armbian/Yocto 系统的 Amlogic NPU 单板环境。当章节以YOLOX目标检测模型为实操案例，开始板端转换推理之前，请务必参照「**3.3.2章节：板端Armbian 安装 amlnn_edge_toolkit**」完成环境部署，再按当前章节流程实现板端模型转换与推理全流程。

4.1 前期准备

为节省板端存储空间，建议直接从该链接：<https://github.com/Amlogic-NN/amlnn-model-playground/tree/main/examples/yolox> 下载所需资源。

Step1:下载模型

```
wget https://pub-8378326bd0fe4b1d9312a3847f6316a2.r2.dev/model_zoo/yolox/yolox_s.onnx
```

Step2:准备量化数据集

因为license限制原因，请自行参考<https://github.com/Amlogic-NN/amlnn-model-playground/tree/main/resource>，准备detection_dataset量化数据集。

Step3: 下载转换脚本和测试文件

- 访问 [amlnn-model-playground/examples/yolox/py](https://github.com/Amlogic-NN/amlnn-model-playground/tree/main/examples/yolox/py)，获取 `export_adla.py` 和 `yolox_inference.py`
- 访问 [amlnn-model-playground/examples/yolox/input](https://github.com/Amlogic-NN/amlnn-model-playground/tree/main/examples/yolox/input)，下载 `test.png`，并保存至本地input目录

4.2 模型转换与推理

在板端环境下，请按照以下步骤完成 YOLOX 模型的 ADLA 转换与推理验证。

Step1: 模型转换 (ONNX to ADLA)

执行 `export_adla.py` 脚本，将 ONNX 模型转换为适配目标平台的 ADLA 格式。

```
python export_adla.py \  
  --onnx yolox_s.onnx \  
  --dataset-path ../../resource/detection_dataset.txt \  
  --target-platform 007 \  
  --adla .
```

Step2: 模型推理

转换完成后，使用生成的 ADLA 模型进行推理测试。

```
python yolox_inference.py \  
  --model-path yolox_s_w8a8.adla \  
  --image-dir input
```

脚本说明：当前 `amlnn-model-playground` 仓库的示例工程中，`export_adla.py`（模型转换脚本）与 `yolox_inference.py`（模型推理脚本）为两个独立文件，二者均基于板端 AMLNN 原生接口开发。可以将二者整合为统一脚本，实现一键跑完ONNX转ADLA+推理验证全流程，具体的整合实现可参考文档 **4.3 章节** 中的示例代码。

4.3 USB摄像头实时采集Demo获取指南

该Demo面向Amlogic开发板实现了YOLOX模型的ADLA格式全流程部署，打通USB摄像头实时取流与端侧目标检测能力，获取方式如下：

Step1: 下载资源包并解压

```
wget https://pub-8378326bd0fe4b1d9312a3847f6316a2.r2.dev/Quick_Start/examples/yolox_camera.tar.gz
tar -zxvf yolox_camera.tar.gz
```

Step2: 部署指引

解压完成后，直接查阅目录内的 `README_CN.md`（中文指引）或 `README.md`（英文指引），即可完成全流程配置与运行。

5 PC端模型转换

本章节以YOLOX目标检测模型为实操案例，系统阐述如何利用amlInn-toolkit工具链将开源模型转换为ADLA格式的板端推理文件。在此基础上，结合第6章与第7章内容，分别介绍3.1节所述的两种推理方式：

- **方式一：**PC端完成模型转换后，通过ADB进行连板推理；
- **方式二：**PC端完成模型转换后，在板端运行C/C++推理程序。

需特别说明，PC端模型转换前，请务必参照「3.3.1章节：Ubuntu PC安装amlInn-toolkit」完成环境部署，再按当前章节流程执行转换操作。

5.1 前期准备

5.1.1 下载模型

```
cd ~/workspace/amlInn-model-playground/examples/yolox/model

wget https://pub-8378326bd0fe4b1d9312a3847f6316a2.r2.dev/model_zoo/yolox/yolox_s.onnx
```

examples 目录下各子目录的 README.md 文件中，除少数因许可证（LICENSE）限制未提供模型获取途径外，其余均详细说明了 ONNX 模型的获取方法或导出步骤。

5.1.2 平台类型确认

在启动ADLA模型转换流程前，必须先完成单板型号确认，获取与硬件完全匹配的 target_platform 参数，避免因平台不兼容导致转换失败或推理异常。当前通过以下两种独立方式完成校验，所有校验结果均可对照下面《Part_number/Serial/target_platform对应关系表》查询得到最终的 target_platform 值。对应关系表如下：

Part_number	Serial	target_platform
C308L / C302X	Serial: 3d0a 010100000000c61349	PRODUCT_PID0XA001
S928X	Serial: 3e0a 0202000000007c404b	PRODUCT_PID0XA002
A311D2	Serial: 360c 01030000000025f904	PRODUCT_PID0XA003
T968D4	Serial: 420a 010100000000daff6e	PRODUCT_PID0XA004
S905X5 / S905D5	Serial: 480a 020100000000682763	PRODUCT_PID0XA005
C302X2	Serial: 4c0a 01030000000055665a	PRODUCT_PID0XA006
A311Y3	Serial: 4d0a 01020000000070e84a	PRODUCT_PID0XA007
C305X2/A123X	Serial: 4f0a 01010000000054c6d1	PRODUCT_PID0XA008

方式一：通过 Part_number 快速确认

直接读取单板硬件表面刻印的Part_number标识，将读取到的编号与对应关系表匹配，即可获取该单板对应的正确 target_platform 参数。

方式二：通过 Serial 号校验确认

获取 Serial 信息：单板正常启动后，通过 ADB 连接设备，执行以下命令可读取 Serial 号：

```
adb shell cat /proc/cpuinfo
# 或
adb shell cat /proc/cpu_chipid
```

匹配规则：

- **首选规则：**优先截取Serial号的前**4位字符**，对照对应关系表完成匹配，获取目标 `target_platform` 值。
- **备选规则：**若前4位无匹配结果，可降级截取Serial号的前**3位字符**完成匹配，最终得到正确的 `target_platform` 参数。

5.2 模型转换

模型转换

```
cd ~/workspace/aml-nn-model-playground/examples/yolox/py

python export_adla.py --onnx ../model/yolox_s.onnx \
    --dataset-path ../../resource/detection_dataset.txt \
    --target-platform 007 \
    --adla ../model
```

参数说明

Parameter	Description
<code>--onnx</code>	<code>.onnx</code> 模型路径
<code>--dataset-path</code>	量化校准数据集。
<code>--target-platform</code>	指定单板类型参数，其取值需与 5.1.2章节 中确认的 <code>target_platform</code> 值的后三位数字保持一致。
<code>--adla</code>	指定adla文件的输出路径，默认值： <code>../model</code>

- 当前转换脚本 `export_adla.py` 已内置多类参数自动适配逻辑，若需确认AMLNN的实际调用参数，请直接参考脚本内AMLNN相关的操作实现部分。
- `detection_dataset.txt`：因为license限制原因，请自行参考<https://github.com/Amlogic-NN/aml-nn-model-playground/tree/main/resource>，准备量化数据集。

转换结果：examples/yolox/model目录下有 `yolox_s_w8a8.adla`模型文件

6 ADB连板推理

进行ADB连板推理之前，请务必参照「[5章节：PC端模型转换](#)」完成ADLA模型文件的转换，再跟随当前章节流程完成ADB连板推理。

ADB连板推理是指Ubuntu PC端通过ADB协议与开发板建立通信链路、传输推理数据，最终在板端NPU完成计算的远程推理方案。该方案依赖板端提前启动nn_server后台代理服务，核心组件定义如下：

- **nnserver**：开发板端后台代理服务，负责PC端Demo与板端NPU的通信。PC端Python程序可通过USB/网络下发推理请求与输入数据，nnserver接收后调用ADLA Runtime完成模型加载、NPU推理并返回结果。在使用PC端Demo、模型验证或连板调试前，必须先在开发板端启动该服务。
- **AML Runtime (libnnsdk.so)**：底层运行时组件，负责模型（计算图）的加载与管理，调度NPU完成推理计算，同时提供I/O缓冲区管理、编译/执行调度等核心能力，为nnserver等上层服务提供底层支撑。

6.1 安装ADB工具

使用ADB连板推理方式，需确保ubuntu PC端已安装ADB。ADB（Android Debug Bridge）是PC端与开发板进行通信和调试的常用工具。Ubuntu PC安装ADB指导如下：

```
sudo apt-get update
sudo apt-get install adb

# 验证安装
adb version

# 连接开发板后，查看设备是否被识别
adb devices

# 预期输出：
#   List of devices attached
#       1234567890   device
```

6.2 启动nnserver

Step1: 获取nnserver文件

```
cd ~/workspace/amlnn-toolkit/
```

根据[\[2.2 系统环境与架构确认\]](#)确认的系统环境，选取对应的nnserver文件。

- **Android 64-bit**: amlnn_toolkit/nnserver/android/lib64/nnserver
- **Android 32-bit**: amlnn_toolkit/nnserver/android/lib32/nnserver
- **Armbian/Yocto 64-bit**: amlnn_toolkit/nnserver/linux/yocto/lib64/nnserver
- **Armbian/Yocto 32-bit**: amlnn_toolkit/nnserver/linux/yocto/lib32/nnserver
- **Buildroot 64-bit**: amlnn_toolkit/nnserver/linux/buildroot/lib64/nnserver
- **Buildroot 32-bit**: amlnn_toolkit/nnserver/linux/buildroot/lib32/nnserver

Step2: 启动nnserver

```
adb shell "mkdir -p /data/nn"
adb push /path/to/nnserver /data/nn/
adb shell "chmod +x /data/nn/nnserver"
adb shell "cd /data/nn && ./nnserver"
```

```
# 如显示类似如下信息，表示nnserver服务成功启动
NNSERVER, vx.x.x, 20xx.xx
Coordinator listening on port 8307
Server listening on port 8308
Server listening on port 8309
Server listening on port 8310
```

6.3 运行 AMLNN Python Demo

进入示例目录并编辑 `yolox_inference.py`，将运行时模式从本地原生模式切换为 NNServer 模式，同时开启性能统计以监控推理耗时。操作步骤如下：

Step1: 进入目录，打开目标文件

```
cd ~/workspace/aml-nn-model-playground/examples/yolox/py
vi yolox_inference.py #定位到 `amlnn.init_runtime()` 调用处
```

Step2: 修改代码

```
# 修改前
# amlnn.init_runtime(mode="native", enable_perf=True)

# 修改后
amlnn.init_runtime(mode="nnserver", enable_perf=True)
```

Step3: 执行推理测试

```
python yolox_inference.py --model-path ../model/yolox_s_w8a8.adla --image-dir
../input
```

Step4: 确认结果

推理结果将自动保存为 `test_result.jpg` 文件。你可以通过 ADB 将该文件拉取至本地查看，具体的检测请参考文档 [7.2.4 章节](#) 图片。

7 板端 C/C++ 推理

在进行板端C/C++推理之前，请务必先参照「**5章节：PC端模型转换**」完成ADLA模型文件的转换，随后再按本章流程，在PC端完成C/C++ Demo的交叉编译，并将生成的可执行文件推送至板端执行。标准AMLNN C/C++ Demo的端到端落地通常涵盖以下三个核心步骤：

- **源码编译**：在PC端利用交叉编译工具链编译Demo源码，生成适配目标板端架构的可执行文件；
- **资源部署**：将生成的可执行文件、转换后的模型文件以及测试图片等依赖资源，通过ADB或网络传输至开发板指定目录；
- **板端执行**：在开发板终端环境中运行Demo程序，校验推理输出结果及性能指标。

7.1 安装编译工具链

7.1.1 安装Cmake

CMake 是跨平台构建系统生成工具，用于生成 Makefile 或工程文件。最低版本要求为 CMake 3.10+，推荐使用 CMake 3.20+。

```
sudo apt-get update
sudo apt-get install cmake

#验证版本
cmake --version

#输出：cmake version 3.x.x
#最低版本要求为 CMake 3.10+，推荐使用 CMake 3.20+

# 如需更高版本，可手动安装
wget https://github.com/Kitware/CMake/releases/download/v3.27.0/cmake-3.27.0-linux-x86_64.sh

sudo bash cmake-3.27.0-linux-x86_64.sh --prefix=/usr/local --skip-license

cmake --version
```

7.1.2 安装交叉编译工具链

依据「**2.2 系统环境与架构确认**」章节中核定的操作系统类型，安装相匹配的交叉编译工具链。

交叉编译工具链是连接开发主机（Host）与目标单板（Target）的核心桥梁，负责在高性能 PC 端生成适配板端 CPU 指令集与 ABI 规范的可执行文件。请根据前序章节确认的系统环境，选择以下对应的安装方案。

7.1.2.1 Android系统：安装与配置NDK

开发板板端为Android系统时，需要安装和配置NDK工具链。

Step1: Android NDK下载链接：<https://github.com/android/ndk/wiki/Unsupported-Downloads>

```
cd ~/workspace

# ===== 下载并配置NDK =====
wget https://dl.google.com/android/repository/android-ndk-r25c-linux.zip
unzip android-ndk-r25c-linux.zip

echo 'export ANDROID_NDK_PATH=$HOME/workspace/android-ndk-r25c' >> ~/.bashrc
echo 'export PATH=$ANDROID_NDK_PATH:$PATH' >> ~/.bashrc
source ~/.bashrc
```

Step2: 验证安装:

```
ndk-build --version
#如果可以正常输出版本信息, 说明 NDK 安装成功
```

7.1.2.2 Buildroot系统: 安装 GCC 交叉编译工具链

开发板板端为 Buildroot 系统时, 需要安装与板端 Release 匹配的 GCC 交叉编译工具链。

注意:

- Buildroot 对工具链版本、glibc、libstdc++ 及 ABI 兼容性较为敏感, 建议优先使用对应板端 Release 随附或明确指定的交叉编译工具链。
- 以下工具链为参考版本。若项目 Release Notes 中另有指定, 请以 Release Notes 为准。
- 开始安装前, 请先按照「2.2 系统环境与架构确认」确认板端为 32 位还是 64 位系统。

For 64-bit System:

```
cd ~/workspace

wget https://developer.arm.com/-/media/Files/downloads/gnu-a/10.3-2021.07/binrel/gcc-arm-10.3-2021.07-x86_64-aarch64-none-linux-gnu.tar.xz

tar -xf gcc-arm-10.3-2021.07-x86_64-aarch64-none-linux-gnu.tar.xz

# 配置工具链环境变量 (仅对当前终端生效)
export BUILDROOT_TOOLCHAIN=$HOME/workspace/gcc-arm-10.3-2021.07-x86_64-aarch64-none-linux-gnu
export PATH=$BUILDROOT_TOOLCHAIN/bin:$PATH

# 验证安装
aarch64-none-linux-gnu-gcc --version
aarch64-none-linux-gnu-g++ --version
```

For 32-bit System:

```
cd ~/workspace

wget https://armkeil.blob.core.windows.net/developer/Files/downloads/gnu-a/10.3-2021.07/binrel/gcc-arm-10.3-2021.07-x86_64-arm-none-linux-gnueabi.tar.xz

tar -xvf gcc-arm-10.3-2021.07-x86_64-arm-none-linux-gnueabi.tar.xz

# 配置工具链环境变量（仅对当前终端生效）
export BUILDROOT_TOOLCHAIN=$HOME/workspace/gcc-arm-10.3-2021.07-x86_64-arm-none-linux-gnueabi
export PATH=$BUILDROOT_TOOLCHAIN/bin:$PATH

# 验证安装
arm-none-linux-gnueabi-gcc --version
arm-none-linux-gnueabi-g++ --version
```

如需使环境变量长期生效，可将对应的 `export` 命令写入 `~/.bashrc`，然后执行：

```
source ~/.bashrc
```

7.1.2.3 Armbian/Yocto系统：安装交叉编译工具链

开发板板端为armbian/yocto系统时，需要安装和配置 对应的交叉编译工具链

For 64-bit System:

```
wget https://pub-8378326bd0fe4b1d9312a3847f6316a2.r2.dev/toolchain/yocto_toolchain/64/poky-glibc-x86_64-meta-toolchain-armv8a-mesont7-an400-5.15-a64-toolchain-4.0.20.sh

./poky-glibc-x86_64-meta-toolchain-armv8a-mesont7-an400-5.15-a64-toolchain-4.0.20.sh
```

For 32-bit System:

```
wget https://pub-8378326bd0fe4b1d9312a3847f6316a2.r2.dev/toolchain/yocto_toolchain/32/poky-glibc-x86_64-amlogic-bsp-armv7at2hf-neon-mesons7-bh201-5.15-a32-toolchain-4.0.20.sh

./poky-glibc-x86_64-amlogic-bsp-armv7at2hf-neon-mesons7-bh201-5.15-a32-toolchain-4.0.20.sh
```

7.2 运行 AMLNN C/C++ demo

7.2.1 Android系统C/C++ demo 编译运行

Step1: 编译 Android 可执行文件

For 64-bit System:

```
cd ~/workspace/am1nn-model-playground/examples/yolox/cpp/

# 默认编译 64 位 arm64-v8a
./build-android.sh

ls build/android/
# yolox_demo
```

For 32-bit System:

```
cd ~/workspace/am1nn-model-playground/examples/yolox/cpp/

./build-android.sh -a armeabi-v7a

ls build/android/
# yolox_demo
```

Step2: 推送并运行

```
cd ~/workspace/am1nn-model-playground/examples/yolox/cpp/

adb shell "mkdir -p /data/local/tmp/yolox"

adb push build/android/yolox_demo /data/local/tmp/yolox/
adb push ../model/yolox_s_w8a8.ad1a /data/local/tmp/yolox/
adb push ../input /data/local/tmp/yolox/
```

板端运行:

```
adb shell
cd /data/local/tmp/yolox
chmod +x yolox_demo

# 64 位 Android
export LD_LIBRARY_PATH=/vendor/lib64:$LD_LIBRARY_PATH

# 32 位 Android 请改为:
# export LD_LIBRARY_PATH=/vendor/lib:$LD_LIBRARY_PATH

# Usage: ./yolox_demo <model_path> <image_dir>
./yolox_demo yolox_s_w8a8.ad1a input
```

7.2.2 Buildroot系统C/C++ demo 编译运行

Step1: 编译 Buildroot 可执行文件

For 64-bit System:

```
export AMLNN_HOME=$HOME/workspace/amlnn-toolkit/amlnn_runtime
export BUILDROOT_TOOLCHAIN=$HOME/workspace/gcc-arm-10.3-2021.07-x86_64-aarch64-
none-linux-gnu
export PATH=$BUILDROOT_TOOLCHAIN/bin:$PATH

cd ~/workspace/amlnn-model-playground/examples/yolox/cpp/

./build-linux.sh

# 默认生成 64 位程序
ls build/linux/64/yolox_demo
```

For 32-bit System:

```
export AMLNN_HOME=$HOME/workspace/amlnn-toolkit/amlnn_runtime
export BUILDROOT_TOOLCHAIN=$HOME/workspace/gcc-arm-10.3-2021.07-x86_64-arm-none-
linux-gnueabi
export PATH=$BUILDROOT_TOOLCHAIN/bin:$PATH

cd ~/workspace/amlnn-model-playground/examples/yolox/cpp/

./build-linux.sh -b 32

ls build/linux/32/yolox_demo
```

Step2: 推送文件

以下以 `/data/local/tmp/yolox` 为示例目录。若当前 Buildroot 系统不存在 `/data/local/tmp`，可替换为 `/root/yolox` 或其他可写目录。

64 位系统：

```
cd ~/workspace/amlnn-model-playground/examples/yolox/cpp/

adb shell "mkdir -p /data/local/tmp/yolox"

adb push build/linux/64/yolox_demo /data/local/tmp/yolox/
adb push ../model/yolox_s_w8a8.adla /data/local/tmp/yolox/
adb push ../input /data/local/tmp/yolox/
```

32 位系统只需将可执行文件路径替换为：

```
adb push build/linux/32/yolox_demo /data/local/tmp/yolox/
```

Step3: 板端运行

```
adb shell
cd /data/local/tmp/yolox
chmod +x yolox_demo

# 检查可执行文件和动态库依赖
file yolox_demo
ldd yolox_demo
```

若 `ldd` 显示 `libnnstdk.so => not found`，请根据板端实际库目录设置：

```
export LD_LIBRARY_PATH=/usr/lib:/usr/lib64:$LD_LIBRARY_PATH
```

运行 Demo：

```
./yolox_demo yolox_s_w8a8.adla input
```

7.2.3 Armbian/Yocto系统C/C++ Demo编译运行

Armbian 与 Yocto 系统均使用 `build-linux.sh` 编译 Linux 版本的 C/C++ Demo。开始编译前，请先完成对应交叉编译工具链的安装与环境配置。

Step1: 配置 AMLNN Runtime 路径

```
export AMLNN_HOME=$HOME/workspace/amlnn-toolkit/amlnn_runtime  
cd ~/workspace/amlnn-model-playground/examples/yolox/cpp
```

Step2: 编译可执行文件

Armbian 64 位系统：

```
./build-linux.sh
```

Armbian 32 位系统：

```
./build-linux.sh -b 32
```

Yocto 64 位系统：

```
./build-linux.sh -m yocto -s /path/to/yocto_sdk_root -t /path/to/toolchain.cmake
```

Yocto 32 位系统：

```
./build-linux.sh -m yocto -b 32 -s /path/to/yocto_sdk_root -t  
/path/to/toolchain.cmake
```

编译产物默认生成在以下目录：

```
# Armbian  
build/linux/64/yolox_demo  
build/linux/32/yolox_demo  
  
# Yocto  
build/yocto/64/yolox_demo  
build/yocto/32/yolox_demo
```

Step3: 推送文件至板端

以下以 64 位系统为例：

```
adb shell "mkdir -p /data/local/tmp/yolox"

# Armbian
adb push build/linux/64/yolox_demo /data/local/tmp/yolox/

# Yocto
# adb push build/yocto/64/yolox_demo /data/local/tmp/yolox/

adb push ../model/yolox_s_w8a8.adla /data/local/tmp/yolox/
adb push ../input /data/local/tmp/yolox/
```

Step4: 板端运行

```
adb shell
cd /data/local/tmp/yolox
chmod +x yolox_demo

# Usage: ./yolox_demo <model_path> <image_dir>
./yolox_demo yolox_s_w8a8.adla input
```

7.2.4 预期结果

默认情况下，检测结果图片将保存至：

```
model_result/<测试图片名称>_result.jpg
```

可通过 ADB 将检测结果拉取至 PC 端查看：

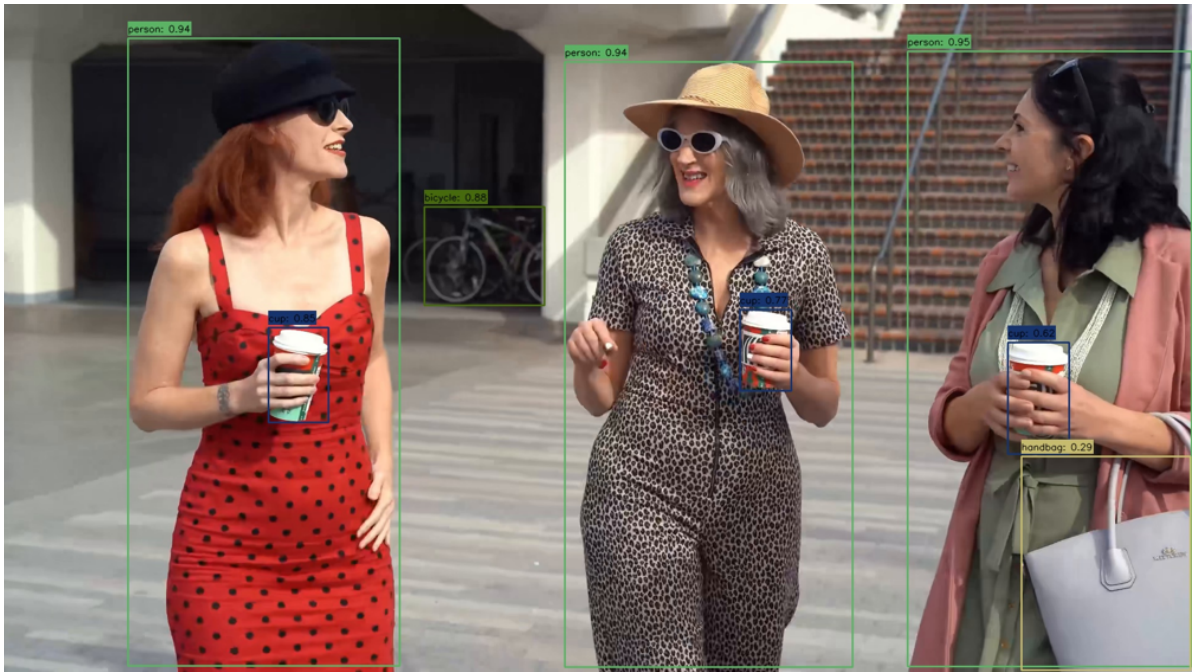
```
adb pull /data/local/tmp/yolox/model_result ./model_result
```

若 Armbian 使用 `/root/yolox` 作为运行目录，则执行：

```
adb pull /root/yolox/model_result ./model_result
```

或使用 SCP：

```
scp -r root@<board-ip>:/root/yolox/model_result ./
```



8 常见问题

8.1 adb devices 无法识别设备

可尝试以下方式排查：

1. 检查数据线连接是否正确，尝试重新插拔数据线、更换计算机 USB 端口或更换数据线。
2. 使用 USB 连接开发板时，确保本地计算机与 Docker 容器中同时仅运行一个 adb server 服务。例如，若需在 Docker 容器中连接开发板，请先在计算机终端执行 `adb kill-server` 终止本地 adb server。
3. 若提示 `command 'adb' not found`，表示系统未安装 adb，需执行 `sudo apt install adb` 进行安装。

8.2 nnserver 服务相关问题

关于 nnserver 的功能说明、部署方式、启动参数、端口配置及常见问题排查，请参考配套文档《10.NNServer.md》，建议在连板调试或服务连接异常时优先查阅